

КОРПОРАТИВНАЯ ИНФОРМАЦИОННАЯ СИСТЕМА

КИС «Управление идеями»

Техническая документация разработчика (API & Architecture Guide)

Дисциплина: Системная и программная инженерия

Разработчики (Группа ИНБО-12-23):

- Мухаяров В. А. (Backend-разработчик)
- Чихалов Г. В. (Руководитель проекта / Архитектор)
- Тимошенко Д. М. (Frontend-разработчик / Аналитик)
- Полиэктов М. А. (Инженер по тестированию / Технический писатель)

Версия документа: 1.0.0-release

Дата генерации: Май, 2026 г.

1. Архитектурный дизайн и системные компоненты

Корпоративная информационная система (КИС) «Управление идеями» построена по многослойной компонентной архитектуре (Tiered/Layered Architecture) с четким разделением зон ответственности (Separation of Concerns). Системное взаимодействие осуществляется по клиент-серверной модели посредством асинхронного REST API протокола.

1.1. Технологический стек слоев системы

Слой архитектуры	Используемые технологии	Назначение и функции
Presentation Layer (Фронтенд)	React.js, Axios, Tailwind CSS, HTML5/ CSS3.	Рендеринг пользовательского веб-интерфейса, управление локальным состоянием, отправка асинхронных HTTP-запросов к API-серверу.
Application Layer (Бэкенд REST API)	Python 3.11+, FastAPI, Pydantic, Uvicorn, Jose (JWT).	Маршрутизация HTTP-запросов, реализация ролевой бизнес-логики (RBAC), сериализация и валидация входящих DTO, авторизация сессий.
Data Access Layer (ORM / Миграции)	SQLAlchemy Core/ORM, Alembic.	Абстрагирование реляционных данных в виде Python-моделей, автоматизированная генерация и проведение миграций структуры таблиц.
Database Layer (Хранилище данных)	PostgreSQL 15+.	Обеспечение транзакционного хранения данных (ACID), выполнение агрегационных SQL-выборок, обеспечение уникальных индексов.

Архитектурный принцип: Бэкенд-приложение спроектировано как Stateless-сервис. Состояние авторизации пользователей не сохраняется в оперативной памяти сервера, а инкапсулируется внутри криптографически подписанных токенов **JWT (JSON Web Tokens)**, передаваемых в HTTP-заголовке `Authorization: Bearer <token>`.

1.2. Схема реляционной базы данных (ER-структура)

Физическая модель данных развернута в СУБД PostgreSQL. Структура состоит из взаимосвязанных таблиц, поддерживающих целостность данных на уровне внешних ключей (Foreign Keys) с каскадными правилами удаления/обновления (ON DELETE CASCADE).

Таблица 1: users (Учетные записи и роли)

Поле	Тип данных	Ограничения	Описание назначения
id	UUID	PRIMARY KEY	Уникальный системный идентификатор пользователя.
username	VARCHAR(50)	UNIQUE, NOT NULL	Уникальный логин учетной записи (корпоративная почта).
password_hash	VARCHAR(255)	NOT NULL	Криптографический хэш пароля (алгоритм bcrypt).
role	VARCHAR(20)	NOT NULL	Роль пользователя в системе: "Сотрудник", "Комитет".

Таблица 2: ideas (Инновационные предложения)

Поле	Тип данных	Ограничения	Описание назначения
id	SERIAL	PRIMARY KEY	Инкрементный уникальный номер идеи.
title	VARCHAR(100)	NOT NULL	Краткое название поданного предложения.
type	VARCHAR(30)	NOT NULL	Категория: "Технологическая", "Организационная".
description	TEXT	NOT NULL	Полное текстовое описание сути инновации.
status	VARCHAR(20)	NOT NULL	Текущий статус: "Голосование", "Реализация", "Реализована", "Архив".
rating	INTEGER	DEFAULT 0	Агрегированный баланс голосов (За - Против).
author_id	UUID	FOREIGN KEY	Ссылка на users (id). Создатель инициативы.

Реализация уникального голосования (Защита от повторов): Запрет на повторное голосование за одну и ту же идею реализован на уровне СУБД с помощью композитного уникального индекса в таблице votes.

2. Спецификация серверного REST API

Все запросы к серверной части КИС проходят через префикс маршрутизации `/api/v1`. Передача параметров осуществляется через JSON-тело запросов (для методов POST/PUT) или Query-параметры (для методов GET).

2.1. Таблица эндпоинтов и маршрутов маршрутизации

Метод	Маршрут (URL)	Права (RBAC)	Ожидаемый результат выполнения
POST	<code>/api/v1/auth/login</code>	Публичный	Авторизация пройдена по логину и паролю. Возвращает JWT токен доступа.
POST	<code>/api/v1/ideas</code>	Авторизован	Идея добавлена в систему ; автору присвоен статус ответственного.
GET	<code>/api/v1/ideas</code>	Авторизован	Список идей отображен с поддержкой фильтрации по query-параметру <code>status</code> .
POST	<code>/api/v1/ideas/{id}/vote</code>	Сотрудник	Голос пользователя зафиксирован ; повторное голосование заблокировано СУБД.
PUT	<code>/api/v1/ideas/{id}/status</code>	Комитет	Решение комитета зарегистрировано . Статус обновлен на Реализация или Архив.
POST	<code>/api/v1/plans</code>	Ответственный	План реализации создан ; участники команды назначены.
POST	<code>/api/v1/feedbacks</code>	Авторизован	Отзыв о реализованной идее добавлен в таблицу обратной связи.

3. Стандарты автоматического документирования Sphinx

Документация исходного кода бэкенд-приложения (Python/FastAPI) генерируется в автоматическом режиме с использованием утилиты **Sphinx** и расширения `sphinx.ext.autodoc`. Для корректной интерпретации структуры кода техническим писателем (Полиэктовым М. А.) утвержден стандарт комментариев `reStructuredText (reST)`.

3.1. Эталонный пример документированного кода (Сервисы)

Ниже представлен пример исходного кода бэкенда, разработанного Мухаяровым В. А., содержащего аннотации типов Pydantic и структурированные комментарии для автоматической генерации Sphinx:

```

from pydantic import BaseModel
from uuid import UUID

class VoteDTO(BaseModel):
    idea_id: int
    value: int

def register_user_vote(user_id: UUID, vote_data: VoteDTO):
    """
    Регистрирует голос сотрудника за выбранную идею в системе.

    Метод выполняет атомарную запись в таблицу `votes`. Если запись для пары
    (user_id, idea_id) уже существует, СУБД сгенерирует IntegrityError,
    и повторное голосование будет заблокировано.

    :param user_id: Уникальный UUID авторизованного пользователя из JWT токена.
    :param vote_data: Валидированный DTO объект.
    :type user_id: UUID
    :type vote_data: VoteDTO

    :raises HTTPException: Возвращает статус 400, если голосование заблокировано.
    :return: Словарь с подтверждением статуса транзакции и обновленным рейтингом.
    :rtype: dict
    """
    pass

```

3.2. Регламент сборки документации разработчика

Для обновления технической документации при добавлении новых модулей, роутеров или моделей базы данных, технический писатель выполняет в корневом каталоге `/docs` следующие CLI-команды:

1. `sphinx-apidoc -o source/ ../backend/` — сканирование структуры проекта и генерация файлов метаописаний модулей `.rst`.
2. `make html` — компиляция исходных текстов и docstrings в набор статических HTML-страниц.

Скомпилированный пакет документации сохраняется в каталоге `/docs/build/html`. Файл `index.html` является точкой входа в навигационную панель описания программных интерфейсов КИС «Управление идеями».